

CERTIPOINT DATABASES HARD MODE MOCK EXAM

Stored Procedures • Triggers • Transactions • Advanced SQL • Normalization

Prepared for Aljun C. Cursiga | Original practice exam

Difficulty: Certiport level to higher-than-Certiport | Total Items: 72 | Recommended Time: 75 minutes

Source basis: Official Certiport reference point: IT Specialist exams are 50 minutes and the Databases exam is listed as 35-45 questions. Certiport describes the Databases certification as validating foundational knowledge of designing and querying relational databases such as MySQL, Microsoft SQL Server, or Oracle. This file is an original training mock and is intentionally harder than the base exam. Official pages checked: Exam lengths and IT Specialist Databases certification page on certiport.pearsonvue.com.

How to Use This Mock Exam

- Recommended hard-mode time limit: 75 minutes. If you want Certiport-style pressure, try 60 minutes.
- Use ANSI SQL reasoning unless the item explicitly says MySQL/MariaDB syntax.
- For stored procedures and triggers, assume MySQL/MariaDB-style syntax when DELIMITER, CALL, OLD, NEW, or SIGNAL is shown.
- Do not use the answer key during your first attempt. Mark hard items and return to them at the end.
- Passing target: 80% or higher. Mastery target: 90% or higher with corrections written for every mistake.

Score Interpretation

Score	Meaning	Action
90-100%	Strong mastery	Practice speed and avoid careless SQL mistakes.
80-89%	Likely ready but still improve	Review every missed explanation and redo all SQL-writing items.
70-79%	Foundation is present but risky	Focus on joins, grouping, constraints, procedures, and triggers.
Below 70%	Needs more drilling	Retake after studying the answer key and rewriting every SQL answer by hand.

Reference Schema for This Exam

```
-- Core academic database
students(student_id PK, student_no UNIQUE, full_name, course_id FK, year_level, status, created_at)
departments(department_id PK, department_name UNIQUE)
courses(course_id PK, course_code UNIQUE, course_name, department_id FK)
instructors(instructor_id PK, full_name, email UNIQUE)
sections(section_id PK, course_id FK, instructor_id FK, semester, school_year, capacity)
enrollments(enrollment_id PK, student_id FK, section_id FK, enrollment_date, grade, status,
            UNIQUE(student_id, section_id))
payments(payment_id PK, student_id FK, amount, paid_at, payment_type)
audit_log(log_id PK, table_name, action_type, old_value, new_value, changed_at, changed_by)

-- Inventory add-on used for trigger questions
products(product_id PK, product_name UNIQUE, stock_qty)
stock_movements(movement_id PK, product_id FK, movement_type, quantity, created_at)
```

Answer Sheet

Use this page while taking the exam. Do not check the answer key yet.

No.	Answer	No.	Answer	No.	Answer	No.	Answer	No.	Answer
1		16		31		46		61	
2		17		32		47		62	
3		18		33		48		63	
4		19		34		49		64	
5		20		35		50		65	
6		21		36		51		66	
7		22		37		52		67	
8		23		38		53		68	
9		24		39		54		69	
10		25		40		55		70	
11		26		41		56		71	
12		27		42		57		72	
13		28		43		58			
14		29		44		59			
15		30		45		60			

Part A - Advanced Multiple Choice and Scenario Questions

Choose the best answer. Some options are intentionally close. Read the schema and wording carefully.

1. A school stores multiple phone numbers inside one column named phone_numbers, separated by commas. Which normalization rule is primarily violated?
 - A. 1NF, because the column contains repeating/non-atomic values
 - B. 2NF, because phone number depends only on part of the key
 - C. 3NF, because phone number depends on another non-key column
 - D. BCNF, because every determinant is a candidate key
2. In enrollments(student_id, section_id, grade), the pair (student_id, section_id) is unique. Which is the best key design if one student can enroll in many sections and one section can have many students?
 - A. Use student_id alone as the primary key
 - B. Use section_id alone as the primary key
 - C. Use a composite key of (student_id, section_id), or a surrogate enrollment_id plus UNIQUE(student_id, section_id)
 - D. Do not define a key because duplicates may be useful
3. A table OrderItems(order_id, product_id, product_name, product_price, quantity) uses (order_id, product_id) as a composite key. product_name and product_price depend only on product_id. What problem exists?
 - A. Partial dependency, violating 2NF
 - B. Transitive dependency, violating 3NF
 - C. No issue because product_id is part of the key
 - D. Referential integrity violation
4. A table Students(student_id, full_name, course_code, course_name, department_name) has student_id as the key. course_name depends on course_code, and department_name depends on course_code. What is the best fix?
 - A. Keep all columns because joins are slow
 - B. Move course and department information into separate related tables
 - C. Make full_name the primary key
 - D. Remove student_id
5. Which data type is most appropriate for money values when exact decimal precision matters?
 - A. FLOAT
 - B. DOUBLE
 - C. DECIMAL(10,2)
 - D. VARCHAR(20)
6. Which constraint prevents an enrollment row from referencing a student_id that does not exist in students?
 - A. PRIMARY KEY
 - B. FOREIGN KEY
 - C. UNIQUE
 - D. DEFAULT
7. You need to allow many students to share the same course_id but prevent duplicate student_no values. Which constraints match the need?
 - A. FOREIGN KEY on student_no, UNIQUE on course_id
 - B. UNIQUE on student_no, FOREIGN KEY on course_id
 - C. PRIMARY KEY on course_id only
 - D. CHECK on student_no only
8. Which statement best describes ON DELETE CASCADE?
 - A. It prevents deleting a parent row when children exist
 - B. It automatically deletes related child rows when the parent row is deleted
 - C. It updates child rows to NULL when parent is deleted
 - D. It creates backup copies before deleting

9. A database user should only run AddEnrollment and GetStudentBalance procedures, not SELECT directly from tables. Which permission approach best follows least privilege?
- A. Grant ALL PRIVILEGES on the database
 - B. Grant SELECT, INSERT, UPDATE, DELETE on every table
 - C. Grant EXECUTE only on the required procedures
 - D. Give the user the admin password
10. Which query correctly lists departments with the number of students in each, including departments with zero students?
- A. SELECT d.department_name, COUNT(s.student_id) FROM departments d INNER JOIN courses c ON d.department_id=c.department_id INNER JOIN students s ON c.course_id=s.course_id GROUP BY d.department_name
 - B. SELECT d.department_name, COUNT(s.student_id) FROM departments d LEFT JOIN courses c ON d.department_id=c.department_id LEFT JOIN students s ON c.course_id=s.course_id GROUP BY d.department_name
 - C. SELECT department_name, COUNT(*) FROM departments GROUP BY student_id
 - D. SELECT d.department_name, SUM(s.student_id) FROM departments d, students s
11. What is wrong with this query?
- ```
SELECT course_id, COUNT(*) AS total
FROM students
WHERE COUNT(*) > 10
GROUP BY course_id;
```
- A. WHERE cannot filter before GROUP BY
  - B. Aggregate filters should use HAVING, not WHERE
  - C. COUNT(\*) cannot be used with GROUP BY
  - D. course\_id must be removed from SELECT
12. Which expression correctly finds rows where grade has no value?
- A. grade = NULL
  - B. grade <> NULL
  - C. grade IS NULL
  - D. grade LIKE NULL
13. A query uses NOT IN with a subquery that may return NULL. What is the danger?
- A. It always runs faster than NOT EXISTS
  - B. It may return no rows because comparisons with NULL become UNKNOWN
  - C. It automatically ignores NULL values
  - D. It converts NULL into zero
14. Which query returns students who have never enrolled in any section?
- A. SELECT s.\* FROM students s INNER JOIN enrollments e ON s.student\_id=e.student\_id WHERE e.student\_id IS NULL
  - B. SELECT s.\* FROM students s LEFT JOIN enrollments e ON s.student\_id=e.student\_id WHERE e.student\_id IS NULL
  - C. SELECT \* FROM students WHERE student\_id = NULL
  - D. SELECT \* FROM enrollments WHERE student\_id NOT NULL
15. Which SQL clause is evaluated logically after GROUP BY and can filter grouped results?
- A. WHERE
  - B. HAVING
  - C. ORDER BY
  - D. SELECT
16. Which query correctly returns each student and the total amount they paid, showing 0 for students with no payments?
- A. SELECT s.full\_name, SUM(p.amount) FROM students s INNER JOIN payments p ON s.student\_id=p.student\_id GROUP BY s.full\_name
  - B. SELECT s.full\_name, COALESCE(SUM(p.amount),0) AS total\_paid FROM students s LEFT JOIN payments p ON s.student\_id=p.student\_id GROUP BY s.student\_id, s.full\_name
  - C. SELECT full\_name, amount FROM students, payments
  - D. SELECT s.full\_name, COUNT(p.amount) FROM students s WHERE p.amount IS NULL
17. Which statement is true about COUNT(\*) and COUNT(column\_name)?

- A. They always return exactly the same value
  - B. COUNT(\*) counts rows; COUNT(column\_name) counts non-NULL values in that column
  - C. COUNT(column\_name) includes NULL values only
  - D. COUNT(\*) can only be used with GROUP BY
18. A view named active\_students shows only students where status='ACTIVE'. Which is the best reason to use a view?
- A. Views permanently copy all data into a new table
  - B. Views can simplify repeated queries and restrict visible columns/rows
  - C. Views remove the need for primary keys
  - D. Views always improve performance
19. Which command changes an existing table structure by adding a column?
- A. UPDATE TABLE students ADD COLUMN middle\_name VARCHAR(50)
  - B. ALTER TABLE students ADD COLUMN middle\_name VARCHAR(50)
  - C. MODIFY students ADD middle\_name
  - D. INSERT COLUMN middle\_name INTO students
20. You need to prevent negative stock quantities in products. Which database-level protection is most direct?
- A. A CHECK constraint such as CHECK(stock\_qty >= 0), if supported
  - B. A VARCHAR data type
  - C. A LEFT JOIN
  - D. A DROP statement
21. What is the main purpose of an index on students(student\_no)?
- A. To guarantee backups
  - B. To speed up searches/filtering by student\_no and optionally enforce uniqueness
  - C. To encrypt the student\_no
  - D. To replace foreign keys
22. Which query is most likely to benefit from an index on enrollments(student\_id)?
- A. SELECT \* FROM enrollments WHERE student\_id = 1001
  - B. SELECT \* FROM enrollments ORDER BY grade DESC
  - C. SELECT COUNT(\*) FROM courses
  - D. DROP TABLE enrollments
23. Which statement best defines a transaction?
- A. A saved SELECT query
  - B. A group of operations treated as one logical unit that should fully succeed or fully fail
  - C. A table relationship
  - D. A column data type
24. In ACID, which property means a transaction's changes are permanent after COMMIT?
- A. Atomicity
  - B. Consistency
  - C. Isolation
  - D. Durability
25. Which sequence is safest when a procedure updates two related tables and both updates must succeed together?
- A. UPDATE first table; COMMIT; UPDATE second table
  - B. START TRANSACTION; update both tables; COMMIT; use ROLLBACK on error
  - C. Disable foreign keys permanently
  - D. Use SELECT only
26. Which stored procedure parameter type sends a value into the procedure but does not return a modified value to the caller?
- A. IN
  - B. OUT
  - C. INOUT

**D. RETURN**

- 27.** What is the role of SELECT ... INTO inside a MySQL stored procedure?
- A. It creates a new table automatically
  - B. It stores query results into local variables
  - C. It deletes selected rows
  - D. It starts a trigger
- 28.** Why is DELIMITER often used before creating a MySQL stored procedure?
- A. To encrypt the procedure
  - B. To let the client distinguish internal semicolons from the end of the CREATE PROCEDURE statement
  - C. To create a foreign key
  - D. To make the procedure run faster
- 29.** Which command executes a MySQL stored procedure named GetStudentById with argument 5?
- A. RUN GetStudentById(5)
  - B. EXECUTE PROCEDURE GetStudentById 5
  - C. CALL GetStudentById(5)
  - D. SELECT PROCEDURE GetStudentById(5)
- 30.** Which stored procedure design is best for AddPayment(p\_student\_id, p\_amount)?
- A. Insert payment without validating student\_id or amount
  - B. Check that student exists, validate p\_amount > 0, then insert within appropriate error handling
  - C. Store amount as text for flexibility
  - D. Drop old payment rows before inserting
- 31.** Which MySQL handler is commonly used to roll back when an SQL error occurs inside a transaction?
- A. DECLARE EXIT HANDLER FOR SQLEXCEPTION
  - B. DECLARE PRIMARY KEY HANDLER
  - C. CREATE ERROR CATCH
  - D. IF ERROR THEN
- 32.** Which is true about triggers?
- A. They are manually executed using CALL
  - B. They automatically run in response to table events such as INSERT, UPDATE, or DELETE
  - C. They replace the need for all constraints
  - D. They can only run once per database
- 33.** A trigger needs to record old and new values after a student's status changes. Which trigger timing/event is most appropriate?
- A. BEFORE INSERT on students
  - B. AFTER UPDATE on students
  - C. BEFORE DELETE on courses
  - D. AFTER SELECT on students
- 34.** Inside a MySQL UPDATE trigger, which pseudo-record refers to the row value before the update?
- A. OLD
  - B. NEW
  - C. PREVIOUS
  - D. BEFORE
- 35.** Inside a MySQL INSERT trigger, which pseudo-record is available?
- A. OLD only
  - B. NEW only
  - C. Both OLD and NEW
  - D. Neither OLD nor NEW
- 36.** Which trigger prevents inserting a stock movement with quantity <= 0?

- A. BEFORE INSERT trigger that checks NEW.quantity and SIGNALs an error when invalid
  - B. AFTER SELECT trigger
  - C. DROP trigger
  - D. A trigger on audit\_log after delete only
37. A procedure and a trigger both contain SQL statements. What is the key difference?
- A. Procedures are called manually or by applications; triggers fire automatically from table events
  - B. Triggers can have IN parameters; procedures cannot
  - C. Procedures only run on DELETE
  - D. Triggers are always faster
38. Which situation is a good use of a trigger?
- A. Automatically inserting an audit record when payment amount is changed
  - B. Showing a dashboard chart in the frontend
  - C. Asking the user for a password
  - D. Replacing all backups
39. Which situation is usually better handled by a stored procedure than a trigger?
- A. A user explicitly requests enrollment, requiring capacity checks, duplicate checks, and an insert
  - B. Log every update to a table
  - C. Set updated\_at automatically for a modified row
  - D. Prevent negative quantity before insert
40. What is the strongest reason not to put too much complex business logic in triggers?
- A. Triggers cannot use SQL
  - B. Triggers can make behavior hidden and harder to debug because they run automatically
  - C. Triggers never affect performance
  - D. Triggers cannot read NEW values
41. The app sends this SQL by concatenating user input. What vulnerability is most likely?
- ```
SELECT * FROM users
WHERE username = '"' + inputUsername + '"'
AND password = '"' + inputPassword + '";
```
- A. SQL injection
 - B. Normalization
 - C. Foreign key cascade
 - D. Index fragmentation
42. Which is the safest application-side way to pass a value to a query or stored procedure?
- A. String concatenation
 - B. Parameterized query / prepared statement
 - C. Manual quote removal only
 - D. Storing SQL in a text file
43. Which backup strategy is strongest for a production database?
- A. No backup if the server is stable
 - B. Manual backup only after problems happen
 - C. Scheduled backups with tested restore process
 - D. Screenshots of tables
44. A query returns duplicate student rows after joining students to enrollments. What is the most likely reason?
- A. Each student can have multiple enrollment records
 - B. JOIN always creates wrong results
 - C. Primary keys are not allowed in joins
 - D. WHERE must never be used with JOIN
45. Which query correctly finds sections that are already full? Assume current enrollment rows with status='ENROLLED' count toward capacity.

- A. `SELECT sec.section_id FROM sections sec JOIN enrollments e ON sec.section_id=e.section_id WHERE COUNT(*) >= sec.capacity`
- B. `SELECT sec.section_id FROM sections sec LEFT JOIN enrollments e ON sec.section_id=e.section_id AND e.status='ENROLLED' GROUP BY sec.section_id, sec.capacity HAVING COUNT(e.enrollment_id) >= sec.capacity`
- C. `SELECT section_id FROM enrollments WHERE status='FULL'`
- D. `SELECT * FROM sections WHERE capacity IS NULL`

46. Which statement about DROP TABLE is correct?

- A. It removes all rows but keeps the table structure
- B. It removes the table object and its data
- C. It only removes duplicate rows
- D. It commits nothing

47. Which command removes all rows from students but keeps the table structure?

- A. `DROP TABLE students`
- B. `DELETE FROM students`
- C. `CREATE TABLE students`
- D. `ALTER TABLE students DROP`

48. A table has a foreign key `course_id` referencing `courses(course_id)`. What should happen before inserting a student with `course_id=10`?

- A. `courses` must already contain `course_id=10`, unless NULL is allowed and used
- B. `students` must have no primary key
- C. `course_id` must be text
- D. The foreign key must be dropped

49. Which SQL creates a table with a primary key and a foreign key?

- A. `CREATE TABLE students (student_id INT PRIMARY KEY, course_id INT, FOREIGN KEY (course_id) REFERENCES courses(course_id));`
- B. `CREATE students TABLE student_id PRIMARY course_id FOREIGN courses;`
- C. `INSERT TABLE students WITH FOREIGN KEY;`
- D. `ALTER FOREIGN KEY students TO courses;`

50. What does this query return?

```
SELECT c.course_code, COUNT(*) AS active_students
FROM courses c
JOIN students s ON s.course_id = c.course_id
WHERE s.status = 'ACTIVE'
GROUP BY c.course_code
HAVING COUNT(*) > 20;
```

- A. All students and all courses even when unrelated
- B. Only courses that have more than 20 active students
- C. One row per active student
- D. Only students with no course

Part B - SQL Writing, Debugging, Procedures, and Triggers

Write or mentally construct the SQL. Compare with the answer key only after finishing the full mock.

51. Which of the following is a transitive dependency example?

- A. student_id -> full_name
- B. student_id -> course_id and course_id -> course_name
- C. (student_id, section_id) -> grade
- D. student_no is unique

52. Which SQL statement updates only active BSCS students to year_level 4?

- A. UPDATE students SET year_level=4
- B. UPDATE students SET year_level=4 WHERE status='ACTIVE' AND course_id = (SELECT course_id FROM courses WHERE course_code='BSCS')
- C. ALTER students year_level=4 WHERE BSCS
- D. INSERT INTO students year_level=4

53. A stored procedure uses an OUT parameter named p_total. Which call pattern is valid in MySQL?

- A. CALL CountStudents(@total); SELECT @total;
- B. SELECT CountStudents() INTO @total;
- C. EXEC CountStudents OUT @total only
- D. DECLARE @total CALL CountStudents

54. Which trigger timing is best for automatically setting NEW.created_at when an INSERT does not provide a value?

- A. BEFORE INSERT
- B. AFTER DELETE
- C. AFTER SELECT
- D. BEFORE DROP

55. Which statement best describes a stored procedure compared with a normal ad hoc query?

- A. A stored procedure is saved in the database and can encapsulate reusable logic
- B. A stored procedure cannot accept parameters
- C. A stored procedure is always just a SELECT statement
- D. A stored procedure is stored only in the application browser

56. Which item is most likely to be tested by a harder-than-Certiport SQL scenario?

- A. Only memorizing the word database
- B. Reading a schema, identifying relationships, and choosing a query that handles NULLs, grouping, and joins correctly
- C. Changing monitor brightness
- D. Writing CSS animations

57. Write a query that returns each course_code and the number of active students in that course. Include only courses with at least 5 active students.

Write your answer in SQL or short explanation form.

58. Write a query that lists students who have paid more than 5000 total. Show student_no, full_name, and total_paid.

Write your answer in SQL or short explanation form.

59. Fix this query. It should show every course even if it has zero students, with the total shown as 0.

```
SELECT c.course_code, COUNT(*) AS total_students
FROM courses c
JOIN students s ON s.course_id = c.course_id
GROUP BY c.course_code;
```

Write your answer in SQL or short explanation form.

60. Create a table named payment_audit with an auto-increment primary key and columns for payment_id, old_amount, new_amount, changed_at.

Write your answer in SQL or short explanation form.

- 61.** Write a MySQL trigger that records payment amount changes into payment_audit after payments.amount is updated.
Write your answer in SQL or short explanation form.
- 62.** Write a MySQL BEFORE INSERT trigger on stock_movements that rejects rows where NEW.quantity <= 0.
Write your answer in SQL or short explanation form.
- 63.** Write a MySQL trigger that updates products.stock_qty after a stock movement is inserted. movement_type='IN' adds quantity; movement_type='OUT' subtracts quantity.
Write your answer in SQL or short explanation form.
- 64.** Write a stored procedure GetStudentPaymentTotal that accepts p_student_id and returns the student's total payment through an OUT parameter p_total.
Write your answer in SQL or short explanation form.
- 65.** Write a stored procedure AddPayment that inserts a payment only when p_amount > 0. If p_amount <= 0, raise an error.
Write your answer in SQL or short explanation form.
- 66.** Improve this procedure by adding a transaction and rollback on SQL exception.
- ```
CREATE PROCEDURE MoveStudentCourse(
 IN p_student_id INT,
 IN p_new_course_id INT
)
BEGIN
 UPDATE students
 SET course_id = p_new_course_id
 WHERE student_id = p_student_id;

 INSERT INTO audit_log(table_name, action_type, old_value, new_value, changed_at, changed_by)
 VALUES('students', 'COURSE_CHANGE', NULL, p_new_course_id, NOW(), CURRENT_USER());
END;
```
- Write your answer in SQL or short explanation form.
- 67.** Write a query to find duplicate emails in instructors. Show email and duplicate\_count. Ignore NULL emails.  
Write your answer in SQL or short explanation form.
- 68.** Write a query to return the top 3 courses by number of enrolled students. Assume MySQL syntax.  
Write your answer in SQL or short explanation form.
- 69.** Fix this trigger. It should set students.created\_at only when the incoming value is NULL.
- ```
CREATE TRIGGER trg_students_created
AFTER INSERT ON students
FOR EACH ROW
BEGIN
    IF created_at = NULL THEN
        created_at = NOW();
    END IF;
END;
```
- Write your answer in SQL or short explanation form.
- 70.** Write a stored procedure CountStudentsByCourse that accepts p_course_code and returns p_total through an OUT parameter.
Write your answer in SQL or short explanation form.
- 71.** Analyze the following design and name the biggest normalization issue: StudentGrades(student_no, full_name, course_code, course_name, instructor_name, grade). One student can take many courses; each course can have one instructor per section.
Write your answer in SQL or short explanation form.
- 72.** Write the safest anti-join query to return students with no payment records, avoiding NOT IN problems with NULL.
Write your answer in SQL or short explanation form.

Answer Key - Part A

1. A school stores multiple phone numbers inside one column named phone_numbers, separated by commas. Which normalization rule is primarily violated?

- A. 1NF, because the column contains repeating/non-atomic values
- B. 2NF, because phone number depends only on part of the key
- C. 3NF, because phone number depends on another non-key column
- D. BCNF, because every determinant is a candidate key

Answer: A

Why: First Normal Form requires atomic values. A comma-separated list in one field is not atomic and makes searching/updating difficult.

2. In enrollments(student_id, section_id, grade), the pair (student_id, section_id) is unique. Which is the best key design if one student can enroll in many sections and one section can have many students?

- A. Use student_id alone as the primary key
- B. Use section_id alone as the primary key
- C. Use a composite key of (student_id, section_id), or a surrogate enrollment_id plus UNIQUE(student_id, section_id)
- D. Do not define a key because duplicates may be useful

Answer: C

Why: The relationship is many-to-many. The natural uniqueness is the combination of student and section. A surrogate key is acceptable only if the pair is still constrained unique.

3. A table OrderItems(order_id, product_id, product_name, product_price, quantity) uses (order_id, product_id) as a composite key. product_name and product_price depend only on product_id. What problem exists?

- A. Partial dependency, violating 2NF
- B. Transitive dependency, violating 3NF
- C. No issue because product_id is part of the key
- D. Referential integrity violation

Answer: A

Why: With a composite key, non-key columns must depend on the whole key. Product attributes depend only on product_id, so they belong in Products.

4. A table Students(student_id, full_name, course_code, course_name, department_name) has student_id as the key. course_name depends on course_code, and department_name depends on course_code. What is the best fix?

- A. Keep all columns because joins are slow
- B. Move course and department information into separate related tables
- C. Make full_name the primary key
- D. Remove student_id

Answer: B

Why: The table has transitive dependencies: student_id -> course_code -> course_name/department. Separate tables reduce update anomalies.

5. Which data type is most appropriate for money values when exact decimal precision matters?

- A. FLOAT
- B. DOUBLE
- C. DECIMAL(10,2)
- D. VARCHAR(20)

Answer: C

Why: DECIMAL stores exact fixed-point values. FLOAT/DOUBLE can introduce rounding issues and VARCHAR prevents numeric operations.

6. Which constraint prevents an enrollment row from referencing a student_id that does not exist in students?

- A. PRIMARY KEY
- B. FOREIGN KEY
- C. UNIQUE
- D. DEFAULT

Answer: B

Why: A foreign key enforces referential integrity between child and parent tables.

7. You need to allow many students to share the same course_id but prevent duplicate student_no values. Which constraints match the need?

- A. FOREIGN KEY on student_no, UNIQUE on course_id
- B. UNIQUE on student_no, FOREIGN KEY on course_id
- C. PRIMARY KEY on course_id only
- D. CHECK on student_no only

Answer: B

Why: student_no should be unique per student. course_id should reference courses and can repeat across many students.

8. Which statement best describes ON DELETE CASCADE?

- A. It prevents deleting a parent row when children exist
- B. It automatically deletes related child rows when the parent row is deleted
- C. It updates child rows to NULL when parent is deleted
- D. It creates backup copies before deleting

Answer: B

Why: CASCADE propagates the parent delete to child records. It must be used carefully because it can remove many rows.

9. A database user should only run AddEnrollment and GetStudentBalance procedures, not SELECT directly from tables. Which permission approach best follows least privilege?

- A. Grant ALL PRIVILEGES on the database
- B. Grant SELECT, INSERT, UPDATE, DELETE on every table
- C. Grant EXECUTE only on the required procedures
- D. Give the user the admin password

Answer: C

Why: Least privilege means granting only the minimum access needed. EXECUTE on procedures can hide direct table access.

10. Which query correctly lists departments with the number of students in each, including departments with zero students?

- A. `SELECT d.department_name, COUNT(s.student_id) FROM departments d INNER JOIN courses c ON d.department_id=c.department_id INNER JOIN students s ON c.course_id=s.course_id GROUP BY d.department_name`
- B. `SELECT d.department_name, COUNT(s.student_id) FROM departments d LEFT JOIN courses c ON d.department_id=c.department_id LEFT JOIN students s ON c.course_id=s.course_id GROUP BY d.department_name`
- C. `SELECT department_name, COUNT(*) FROM departments GROUP BY student_id`
- D. `SELECT d.department_name, SUM(s.student_id) FROM departments d, students s`

Answer: B

Why: LEFT JOIN keeps departments even when no matching courses or students exist. COUNT(s.student_id) counts only matching student rows.

11. What is wrong with this query?

```
SELECT course_id, COUNT(*) AS total
FROM students
WHERE COUNT(*) > 10
```

GROUP BY course_id;

- A. WHERE cannot filter before GROUP BY
- B. Aggregate filters should use HAVING, not WHERE
- C. COUNT(*) cannot be used with GROUP BY
- D. course_id must be removed from SELECT

Answer: B

Why: WHERE filters rows before grouping. Conditions on aggregate results such as COUNT(*) > 10 belong in HAVING.

```
SELECT course_id, COUNT(*) AS total
FROM students
GROUP BY course_id
HAVING COUNT(*) > 10;
```

12. Which expression correctly finds rows where grade has no value?

- A. grade = NULL
- B. grade <> NULL
- C. grade IS NULL
- D. grade LIKE NULL

Answer: C

Why: NULL means unknown/missing. SQL uses IS NULL and IS NOT NULL for NULL checks.

13. A query uses NOT IN with a subquery that may return NULL. What is the danger?

- A. It always runs faster than NOT EXISTS
- B. It may return no rows because comparisons with NULL become UNKNOWN
- C. It automatically ignores NULL values
- D. It converts NULL into zero

Answer: B

Why: NOT IN can behave unexpectedly when the list contains NULL. NOT EXISTS is often safer for anti-joins.

14. Which query returns students who have never enrolled in any section?

- A. SELECT s.* FROM students s INNER JOIN enrollments e ON s.student_id=e.student_id WHERE e.student_id IS NULL
- B. SELECT s.* FROM students s LEFT JOIN enrollments e ON s.student_id=e.student_id WHERE e.student_id IS NULL
- C. SELECT * FROM students WHERE student_id = NULL
- D. SELECT * FROM enrollments WHERE student_id NOT NULL

Answer: B

Why: A LEFT JOIN returns all students. Rows with no matching enrollment have NULL in the enrollment side.

15. Which SQL clause is evaluated logically after GROUP BY and can filter grouped results?

- A. WHERE
- B. HAVING
- C. ORDER BY
- D. SELECT

Answer: B

Why: HAVING filters groups. WHERE filters individual rows before grouping.

16. Which query correctly returns each student and the total amount they paid, showing 0 for students with no payments?

- A. SELECT s.full_name, SUM(p.amount) FROM students s INNER JOIN payments p ON s.student_id=p.student_id GROUP BY s.full_name
- B. SELECT s.full_name, COALESCE(SUM(p.amount),0) AS total_paid FROM students s LEFT JOIN payments p ON s.student_id=p.student_id GROUP BY s.student_id, s.full_name
- C. SELECT full_name, amount FROM students, payments

D. SELECT s.full_name, COUNT(p.amount) FROM students s WHERE p.amount IS NULL

Answer: B

Why: LEFT JOIN keeps unpaid students. COALESCE turns NULL sum into 0. Grouping by ID and name avoids ambiguity.

17. Which statement is true about COUNT(*) and COUNT(column_name)?

- A. They always return exactly the same value
- B. COUNT(*) counts rows; COUNT(column_name) counts non-NULL values in that column
- C. COUNT(column_name) includes NULL values only
- D. COUNT(*) can only be used with GROUP BY

Answer: B

Why: COUNT(*) counts all rows in the group. COUNT(column) ignores NULL in that column.

18. A view named active_students shows only students where status='ACTIVE'. Which is the best reason to use a view?

- A. Views permanently copy all data into a new table
- B. Views can simplify repeated queries and restrict visible columns/rows
- C. Views remove the need for primary keys
- D. Views always improve performance

Answer: B

Why: A view is a stored query. It can simplify access and help expose only selected data. It is not always faster and usually does not copy data.

19. Which command changes an existing table structure by adding a column?

- A. UPDATE TABLE students ADD COLUMN middle_name VARCHAR(50)
- B. ALTER TABLE students ADD COLUMN middle_name VARCHAR(50)
- C. MODIFY students ADD middle_name
- D. INSERT COLUMN middle_name INTO students

Answer: B

Why: ALTER TABLE is used for schema changes such as adding, modifying, or dropping columns.

20. You need to prevent negative stock quantities in products. Which database-level protection is most direct?

- A. A CHECK constraint such as CHECK(stock_qty >= 0), if supported
- B. A VARCHAR data type
- C. A LEFT JOIN
- D. A DROP statement

Answer: A

Why: A CHECK constraint expresses a rule that the database can enforce regardless of the application.

21. What is the main purpose of an index on students(student_no)?

- A. To guarantee backups
- B. To speed up searches/filtering by student_no and optionally enforce uniqueness
- C. To encrypt the student_no
- D. To replace foreign keys

Answer: B

Why: Indexes help the DBMS find rows faster. A unique index/constraint can also prevent duplicates.

22. Which query is most likely to benefit from an index on enrollments(student_id)?

- A. SELECT * FROM enrollments WHERE student_id = 1001
- B. SELECT * FROM enrollments ORDER BY grade DESC

C. SELECT COUNT(*) FROM courses

D. DROP TABLE enrollments

Answer: A

Why: Filtering by the indexed column can use the index to locate matching rows more efficiently.

23. Which statement best defines a transaction?

A. A saved SELECT query

B. A group of operations treated as one logical unit that should fully succeed or fully fail

C. A table relationship

D. A column data type

Answer: B

Why: Transactions protect consistency. COMMIT saves the unit; ROLLBACK undoes it if needed.

24. In ACID, which property means a transaction's changes are permanent after COMMIT?

A. Atomicity

B. Consistency

C. Isolation

D. Durability

Answer: D

Why: Durability means committed data remains saved even after failures, subject to DBMS logging/recovery.

25. Which sequence is safest when a procedure updates two related tables and both updates must succeed together?

A. UPDATE first table; COMMIT; UPDATE second table

B. START TRANSACTION; update both tables; COMMIT; use ROLLBACK on error

C. Disable foreign keys permanently

D. Use SELECT only

Answer: B

Why: A transaction with error handling prevents partial updates from becoming permanent.

26. Which stored procedure parameter type sends a value into the procedure but does not return a modified value to the caller?

A. IN

B. OUT

C. INOUT

D. RETURN

Answer: A

Why: IN parameters are inputs. OUT returns values. INOUT can be both input and output.

27. What is the role of SELECT ... INTO inside a MySQL stored procedure?

A. It creates a new table automatically

B. It stores query results into local variables

C. It deletes selected rows

D. It starts a trigger

Answer: B

Why: SELECT ... INTO variable_name is commonly used to assign query output to procedure variables.

28. Why is DELIMITER often used before creating a MySQL stored procedure?

A. To encrypt the procedure

- B. To let the client distinguish internal semicolons from the end of the CREATE PROCEDURE statement
- C. To create a foreign key
- D. To make the procedure run faster

Answer: B

Why: Procedures contain semicolons inside BEGIN...END. DELIMITER temporarily changes the command terminator used by the client.

29. Which command executes a MySQL stored procedure named GetStudentById with argument 5?

- A. RUN GetStudentById(5)
- B. EXECUTE PROCEDURE GetStudentById 5
- C. CALL GetStudentById(5)
- D. SELECT PROCEDURE GetStudentById(5)

Answer: C

Why: MySQL uses CALL procedure_name(arguments). SQL Server commonly uses EXEC.

30. Which stored procedure design is best for AddPayment(p_student_id, p_amount)?

- A. Insert payment without validating student_id or amount
- B. Check that student exists, validate p_amount > 0, then insert within appropriate error handling
- C. Store amount as text for flexibility
- D. Drop old payment rows before inserting

Answer: B

Why: Good procedures validate inputs and protect data integrity. Payment amount should be positive and linked to an existing student.

31. Which MySQL handler is commonly used to roll back when an SQL error occurs inside a transaction?

- A. DECLARE EXIT HANDLER FOR SQLEXCEPTION
- B. DECLARE PRIMARY KEY HANDLER
- C. CREATE ERROR CATCH
- D. IF ERROR THEN

Answer: A

Why: MySQL supports handlers such as DECLARE EXIT HANDLER FOR SQLEXCEPTION to define what happens on errors.

32. Which is true about triggers?

- A. They are manually executed using CALL
- B. They automatically run in response to table events such as INSERT, UPDATE, or DELETE
- C. They replace the need for all constraints
- D. They can only run once per database

Answer: B

Why: Triggers are event-driven database objects. They fire automatically when their associated event occurs.

33. A trigger needs to record old and new values after a student's status changes. Which trigger timing/event is most appropriate?

- A. BEFORE INSERT on students
- B. AFTER UPDATE on students
- C. BEFORE DELETE on courses
- D. AFTER SELECT on students

Answer: B

Why: The change has already occurred after UPDATE, and both OLD.status and NEW.status are available for logging.

34. Inside a MySQL UPDATE trigger, which pseudo-record refers to the row value before the update?

- A. OLD
- B. NEW
- C. PREVIOUS
- D. BEFORE

Answer: A

Why: OLD contains pre-change values. NEW contains incoming/changed values for INSERT/UPDATE.

35. Inside a MySQL INSERT trigger, which pseudo-record is available?

- A. OLD only
- B. NEW only
- C. Both OLD and NEW
- D. Neither OLD nor NEW

Answer: B

Why: For INSERT, there is no old row. NEW contains the values being inserted.

36. Which trigger prevents inserting a stock movement with quantity <= 0?

- A. BEFORE INSERT trigger that checks NEW.quantity and SIGNALs an error when invalid
- B. AFTER SELECT trigger
- C. DROP trigger
- D. A trigger on audit_log after delete only

Answer: A

Why: BEFORE INSERT can validate NEW values before the row is saved. SIGNAL can stop the operation.

37. A procedure and a trigger both contain SQL statements. What is the key difference?

- A. Procedures are called manually or by applications; triggers fire automatically from table events
- B. Triggers can have IN parameters; procedures cannot
- C. Procedures only run on DELETE
- D. Triggers are always faster

Answer: A

Why: Procedures are explicitly executed. Triggers are attached to table events.

38. Which situation is a good use of a trigger?

- A. Automatically inserting an audit record when payment amount is changed
- B. Showing a dashboard chart in the frontend
- C. Asking the user for a password
- D. Replacing all backups

Answer: A

Why: Audit logging on data changes is a classic trigger use case because it must happen consistently whenever the table changes.

39. Which situation is usually better handled by a stored procedure than a trigger?

- A. A user explicitly requests enrollment, requiring capacity checks, duplicate checks, and an insert
- B. Log every update to a table
- C. Set updated_at automatically for a modified row
- D. Prevent negative quantity before insert

Answer: A

Why: A user-requested multi-step business operation is a good procedure. Triggers are better for automatic reactions to table changes.

40. What is the strongest reason not to put too much complex business logic in triggers?

- A. Triggers cannot use SQL
- B. Triggers can make behavior hidden and harder to debug because they run automatically
- C. Triggers never affect performance
- D. Triggers cannot read NEW values

Answer: B

Why: Triggers can surprise developers if undocumented and may cause hard-to-trace side effects.

41. The app sends this SQL by concatenating user input. What vulnerability is most likely?

```
SELECT * FROM users
WHERE username = '"' + inputUsername + '"'
AND password = '"' + inputPassword + '');
```

- A. SQL injection
- B. Normalization
- C. Foreign key cascade
- D. Index fragmentation

Answer: A

Why: Concatenating raw user input into SQL can allow attackers to change the query. Parameterized queries are safer.

42. Which is the safest application-side way to pass a value to a query or stored procedure?

- A. String concatenation
- B. Parameterized query / prepared statement
- C. Manual quote removal only
- D. Storing SQL in a text file

Answer: B

Why: Prepared statements separate SQL structure from data values, reducing injection risk.

43. Which backup strategy is strongest for a production database?

- A. No backup if the server is stable
- B. Manual backup only after problems happen
- C. Scheduled backups with tested restore process
- D. Screenshots of tables

Answer: C

Why: A backup is only trustworthy if it is scheduled and restore-tested.

44. A query returns duplicate student rows after joining students to enrollments. What is the most likely reason?

- A. Each student can have multiple enrollment records
- B. JOIN always creates wrong results
- C. Primary keys are not allowed in joins
- D. WHERE must never be used with JOIN

Answer: A

Why: One-to-many joins repeat the parent row for each matching child row. This may be expected, or you may need aggregation/DISTINCT.

45. Which query correctly finds sections that are already full? Assume current enrollment rows with status='ENROLLED' count toward capacity.

- A. `SELECT sec.section_id FROM sections sec JOIN enrollments e ON sec.section_id=e.section_id WHERE COUNT(*) >= sec.capacity`

- B. `SELECT sec.section_id FROM sections sec LEFT JOIN enrollments e ON sec.section_id=e.section_id AND e.status='ENROLLED' GROUP BY sec.section_id, sec.capacity HAVING COUNT(e.enrollment_id) >= sec.capacity`
- C. `SELECT section_id FROM enrollments WHERE status='FULL'`
- D. `SELECT * FROM sections WHERE capacity IS NULL`

Answer: B

Why: The count must be grouped per section and compared in HAVING. LEFT JOIN also handles zero-enrollment sections.

46. Which statement about DROP TABLE is correct?

- A. It removes all rows but keeps the table structure
- B. It removes the table object and its data
- C. It only removes duplicate rows
- D. It commits nothing

Answer: B

Why: DROP TABLE deletes the table definition and data. DELETE removes rows while keeping the table.

47. Which command removes all rows from students but keeps the table structure?

- A. `DROP TABLE students`
- B. `DELETE FROM students`
- C. `CREATE TABLE students`
- D. `ALTER TABLE students DROP`

Answer: B

Why: DELETE removes rows. Depending on DBMS, TRUNCATE can also remove rows but has different transaction/permission behavior.

48. A table has a foreign key course_id referencing courses(course_id). What should happen before inserting a student with course_id=10?

- A. courses must already contain course_id=10, unless NULL is allowed and used
- B. students must have no primary key
- C. course_id must be text
- D. The foreign key must be dropped

Answer: A

Why: A child foreign key value must match an existing parent key, unless the FK column is nullable and set to NULL.

49. Which SQL creates a table with a primary key and a foreign key?

- A. `CREATE TABLE students (student_id INT PRIMARY KEY, course_id INT, FOREIGN KEY (course_id) REFERENCES courses(course_id));`
- B. `CREATE students TABLE student_id PRIMARY course_id FOREIGN courses;`
- C. `INSERT TABLE students WITH FOREIGN KEY;`
- D. `ALTER FOREIGN KEY students TO courses;`

Answer: A

Why: It uses valid CREATE TABLE syntax with a primary key and a foreign key referencing the parent table.

50. What does this query return?

```
SELECT c.course_code, COUNT(*) AS active_students
FROM courses c
JOIN students s ON s.course_id = c.course_id
WHERE s.status = 'ACTIVE'
GROUP BY c.course_code
HAVING COUNT(*) > 20;
```

- A. All students and all courses even when unrelated

- B. Only courses that have more than 20 active students
- C. One row per active student
- D. Only students with no course

Answer: B

Why: The query groups active students by course and keeps only groups whose count is greater than 20.

Answer Key - Part B

51. Which of the following is a transitive dependency example?

- A. student_id -> full_name
- B. student_id -> course_id and course_id -> course_name
- C. (student_id, section_id) -> grade
- D. student_no is unique

Answer: B

Why: A non-key attribute depends on another non-key determinant through the key: student_id determines course_id, then course_id determines course_name.

52. Which SQL statement updates only active BSCS students to year_level 4?

- A. UPDATE students SET year_level=4
- B. UPDATE students SET year_level=4 WHERE status='ACTIVE' AND course_id = (SELECT course_id FROM courses WHERE course_code='BSCS')
- C. ALTER students year_level=4 WHERE BSCS
- D. INSERT INTO students year_level=4

Answer: B

Why: The WHERE clause limits the update. Without it, every row may be changed.

53. A stored procedure uses an OUT parameter named p_total. Which call pattern is valid in MySQL?

- A. CALL CountStudents(@total); SELECT @total;
- B. SELECT CountStudents() INTO @total;
- C. EXEC CountStudents OUT @total only
- D. DECLARE @total CALL CountStudents

Answer: A

Why: MySQL often uses session variables such as @total to receive OUT parameter values from CALL.

54. Which trigger timing is best for automatically setting NEW.created_at when an INSERT does not provide a value?

- A. BEFORE INSERT
- B. AFTER DELETE
- C. AFTER SELECT
- D. BEFORE DROP

Answer: A

Why: A BEFORE INSERT trigger can modify NEW values before they are written.

55. Which statement best describes a stored procedure compared with a normal ad hoc query?

- A. A stored procedure is saved in the database and can encapsulate reusable logic
- B. A stored procedure cannot accept parameters
- C. A stored procedure is always just a SELECT statement
- D. A stored procedure is stored only in the application browser

Answer: A

Why: Stored procedures are database objects that can include multiple SQL statements, variables, conditions, and transactions.

56. Which item is most likely to be tested by a harder-than-Certiport SQL scenario?

- A. Only memorizing the word database
- B. Reading a schema, identifying relationships, and choosing a query that handles NULLs, grouping, and joins correctly
- C. Changing monitor brightness
- D. Writing CSS animations

Answer: B

Why: Hard database exams test schema understanding plus query behavior, especially joins, NULLs, aggregation, and constraints.

57. Write a query that returns each course_code and the number of active students in that course. Include only courses with at least 5 active students.

Write your answer in SQL or short explanation form.

Answer: Use JOIN, WHERE, GROUP BY, and HAVING.

Why: Rows must be filtered to active students before grouping; the count threshold is a group condition.

```
SELECT c.course_code, COUNT(s.student_id) AS active_student_count
FROM courses c
JOIN students s ON s.course_id = c.course_id
WHERE s.status = 'ACTIVE'
GROUP BY c.course_code
HAVING COUNT(s.student_id) >= 5;
```

58. Write a query that lists students who have paid more than 5000 total. Show student_no, full_name, and total_paid.

Write your answer in SQL or short explanation form.

Answer: Group payments by student and filter with HAVING SUM(p.amount) > 5000.

Why: SUM is an aggregate, so the filter belongs in HAVING. Join students to payments so names and student numbers are available.

```
SELECT s.student_no, s.full_name, SUM(p.amount) AS total_paid
FROM students s
JOIN payments p ON p.student_id = s.student_id
GROUP BY s.student_id, s.student_no, s.full_name
HAVING SUM(p.amount) > 5000;
```

59. Fix this query. It should show every course even if it has zero students, with the total shown as 0.

```
SELECT c.course_code, COUNT(*) AS total_students
FROM courses c
JOIN students s ON s.course_id = c.course_id
GROUP BY c.course_code;
```

Write your answer in SQL or short explanation form.

Answer: Use LEFT JOIN from courses to students and COUNT(s.student_id).

Why: INNER JOIN removes courses with no students. COUNT(s.student_id) returns 0 for no matches; COUNT(*) would count the course row.

```
SELECT c.course_code, COUNT(s.student_id) AS total_students
FROM courses c
LEFT JOIN students s ON s.course_id = c.course_id
GROUP BY c.course_id, c.course_code;
```

60. Create a table named payment_audit with an auto-increment primary key and columns for payment_id, old_amount, new_amount, changed_at.

Write your answer in SQL or short explanation form.

Answer: A valid CREATE TABLE statement with numeric amount fields and timestamp.

Why: The audit table needs its own primary key and enough fields to record the changed payment values.

```
CREATE TABLE payment_audit (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    payment_id INT NOT NULL,
    old_amount DECIMAL(10,2),
    new_amount DECIMAL(10,2),
    changed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

61. Write a MySQL trigger that records payment amount changes into payment_audit after payments.amount is updated.

Write your answer in SQL or short explanation form.

Answer: AFTER UPDATE trigger using OLD.amount and NEW.amount, with an IF check to log only actual amount changes.

Why: AFTER UPDATE is appropriate because the payment row has changed. OLD and NEW allow logging before and after values.

```
DELIMITER //
```

```
CREATE TRIGGER trg_payments_amount_audit
AFTER UPDATE ON payments
FOR EACH ROW
BEGIN
    IF OLD.amount <> NEW.amount THEN
        INSERT INTO payment_audit(payment_id, old_amount, new_amount, changed_at)
        VALUES(OLD.payment_id, OLD.amount, NEW.amount, NOW());
    END IF;
END //
```

```
DELIMITER ;
```

62. Write a MySQL BEFORE INSERT trigger on stock_movements that rejects rows where NEW.quantity <= 0.

Write your answer in SQL or short explanation form.

Answer: Use SIGNAL SQLSTATE '45000' inside BEFORE INSERT.

Why: BEFORE INSERT can stop invalid data before it is saved. SQLSTATE '45000' is commonly used for user-defined exceptions.

```
DELIMITER //
```

```
CREATE TRIGGER trg_stock_movement_positive_qty
BEFORE INSERT ON stock_movements
FOR EACH ROW
BEGIN
    IF NEW.quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Quantity must be greater than zero';
    END IF;
END //
```

```
DELIMITER ;
```

63. Write a MySQL trigger that updates products.stock_qty after a stock movement is inserted. movement_type='IN' adds quantity; movement_type='OUT' subtracts quantity.

Write your answer in SQL or short explanation form.

Answer: Use AFTER INSERT ON stock_movements and update products based on NEW.movement_type.

Why: The movement has already been inserted, so the trigger can update the related product stock. A production version should also prevent negative stock.

```
DELIMITER //
```

```
CREATE TRIGGER trg_apply_stock_movement
AFTER INSERT ON stock_movements
FOR EACH ROW
BEGIN
    IF NEW.movement_type = 'IN' THEN
        UPDATE products
        SET stock_qty = stock_qty + NEW.quantity
        WHERE product_id = NEW.product_id;
    ELSEIF NEW.movement_type = 'OUT' THEN
        UPDATE products
        SET stock_qty = stock_qty - NEW.quantity
        WHERE product_id = NEW.product_id;
    END IF;
END //
```

```
DELIMITER ;
```

64. Write a stored procedure GetStudentPaymentTotal that accepts p_student_id and returns the student's total payment through an OUT parameter p_total.

Write your answer in SQL or short explanation form.

Answer: Use SELECT COALESCE(SUM(amount),0) INTO p_total.

Why: COALESCE handles students with no payments by returning 0 instead of NULL.

```
DELIMITER //
```

```
CREATE PROCEDURE GetStudentPaymentTotal(  
    IN p_student_id INT,  
    OUT p_total DECIMAL(10,2)  
)  
BEGIN  
    SELECT COALESCE(SUM(amount), 0)  
    INTO p_total  
    FROM payments  
    WHERE student_id = p_student_id;  
END //
```

```
DELIMITER ;
```

```
-- Example call:  
CALL GetStudentPaymentTotal(1001, @total);  
SELECT @total AS total_paid;
```

65. Write a stored procedure AddPayment that inserts a payment only when p_amount > 0. If p_amount <= 0, raise an error.

Write your answer in SQL or short explanation form.

Answer: Use IF and SIGNAL before INSERT.

Why: Procedures should validate business rules before changing data.

```
DELIMITER //
```

```
CREATE PROCEDURE AddPayment(  
    IN p_student_id INT,  
    IN p_amount DECIMAL(10,2),  
    IN p_payment_type VARCHAR(30)  
)  
BEGIN  
    IF p_amount <= 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Payment amount must be greater than zero';  
    END IF;  
  
    INSERT INTO payments(student_id, amount, paid_at, payment_type)  
    VALUES(p_student_id, p_amount, NOW(), p_payment_type);  
END //
```

```
DELIMITER ;
```

66. Improve this procedure by adding a transaction and rollback on SQL exception.

```
CREATE PROCEDURE MoveStudentCourse(  
    IN p_student_id INT,  
    IN p_new_course_id INT  
)  
BEGIN  
    UPDATE students  
    SET course_id = p_new_course_id  
    WHERE student_id = p_student_id;  
  
    INSERT INTO audit_log(table_name, action_type, old_value, new_value, changed_at, changed_by)  
    VALUES('students', 'COURSE_CHANGE', NULL, p_new_course_id, NOW(), CURRENT_USER());  
END;
```


Write your answer in SQL or short explanation form.

Answer: Add `DECLARE EXIT HANDLER FOR SQLEXCEPTION, START TRANSACTION, COMMIT, and ROLLBACK` inside the handler.

Why: When a procedure performs multiple related writes, a transaction prevents partial success.

```
DELIMITER //
```

```
CREATE PROCEDURE MoveStudentCourse(
    IN p_student_id INT,
    IN p_new_course_id INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    UPDATE students
    SET course_id = p_new_course_id
    WHERE student_id = p_student_id;

    INSERT INTO audit_log(table_name, action_type, old_value, new_value, changed_at, changed_by)
    VALUES('students', 'COURSE_CHANGE', NULL, p_new_course_id, NOW(), CURRENT_USER());

    COMMIT;
END //
```

```
DELIMITER ;
```

67. Write a query to find duplicate emails in instructors. Show email and duplicate_count. Ignore NULL emails.

Write your answer in SQL or short explanation form.

Answer: Group by email, `WHERE email IS NOT NULL, HAVING COUNT(*) > 1`.

Why: `WHERE` removes NULL before grouping. `HAVING` filters duplicate groups.

```
SELECT email, COUNT(*) AS duplicate_count
FROM instructors
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

68. Write a query to return the top 3 courses by number of enrolled students. Assume MySQL syntax.

Write your answer in SQL or short explanation form.

Answer: Use `GROUP BY, ORDER BY` count descending, `LIMIT 3`.

Why: A ranking-style query often combines aggregation with ordering and limiting.

```
SELECT c.course_code, COUNT(e.enrollment_id) AS enrolled_count
FROM courses c
JOIN sections sec ON sec.course_id = c.course_id
JOIN enrollments e ON e.section_id = sec.section_id
WHERE e.status = 'ENROLLED'
GROUP BY c.course_id, c.course_code
ORDER BY enrolled_count DESC
LIMIT 3;
```

69. Fix this trigger. It should set `students.created_at` only when the incoming value is NULL.

```
CREATE TRIGGER trg_students_created
AFTER INSERT ON students
FOR EACH ROW
BEGIN
```

```

        IF created_at = NULL THEN
            created_at = NOW();
        END IF;
    END;

```

Write your answer in SQL or short explanation form.

Answer: Use BEFORE INSERT and SET NEW.created_at = NOW() when NEW.created_at IS NULL.

Why: INSERT triggers can modify NEW values before the row is stored. The original trigger uses invalid assignment syntax and wrong timing.

```

DELIMITER //

CREATE TRIGGER trg_students_created
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
    IF NEW.created_at IS NULL THEN
        SET NEW.created_at = NOW();
    END IF;
END //

DELIMITER ;

```

70. Write a stored procedure CountStudentsByCourse that accepts p_course_code and returns p_total through an OUT parameter.

Write your answer in SQL or short explanation form.

Answer: Join courses to students and count matching students.

Why: The procedure accepts a human-friendly course code but must count rows through the related course_id.

```

DELIMITER //

CREATE PROCEDURE CountStudentsByCourse(
    IN p_course_code VARCHAR(20),
    OUT p_total INT
)
BEGIN
    SELECT COUNT(s.student_id)
    INTO p_total
    FROM courses c
    LEFT JOIN students s ON s.course_id = c.course_id
    WHERE c.course_code = p_course_code;
END //

DELIMITER ;

```

71. Analyze the following design and name the biggest normalization issue: StudentGrades(student_no, full_name, course_code, course_name, instructor_name, grade). One student can take many courses; each course can have one instructor per section.

Write your answer in SQL or short explanation form.

Answer: It mixes student, course, instructor/section, and enrollment/grade facts in one table; it causes redundancy and update anomalies.

Why: A normalized design would separate Students, Courses, Instructors, Sections, and Enrollments/Grades. Otherwise course_name and instructor data repeat for many students.

72. Write the safest anti-join query to return students with no payment records, avoiding NOT IN problems with NULL.

Write your answer in SQL or short explanation form.

Answer: Use NOT EXISTS or LEFT JOIN IS NULL. NOT EXISTS is robust.

Why: NOT EXISTS checks absence of matching rows and is not confused by NULL values returned from a subquery.

```

SELECT s.student_id, s.student_no, s.full_name

```

```
FROM students s
WHERE NOT EXISTS (
    SELECT 1
    FROM payments p
    WHERE p.student_id = s.student_id
);
```

High-Yield Review Checklist

- Can I identify 1NF, 2NF, and 3NF violations from a scenario?
- Can I choose the correct primary key, composite key, and foreign key design?
- Can I explain LEFT JOIN vs INNER JOIN using missing records?
- Can I use GROUP BY and HAVING correctly?
- Can I handle NULL using IS NULL, COALESCE, and NOT EXISTS?
- Can I write CREATE TABLE with PK, FK, UNIQUE, NOT NULL, and CHECK?
- Can I explain transactions, COMMIT, ROLLBACK, and ACID?
- Can I write a stored procedure with IN and OUT parameters?
- Can I use DECLARE EXIT HANDLER FOR SQLEXCEPTION for rollback?
- Can I write triggers using BEFORE/AFTER and OLD/NEW correctly?
- Can I explain when NOT to use triggers because of hidden side effects?
- Can I describe least privilege, GRANT EXECUTE, backups, and restore testing?

3-Day Hard Mode Drill Plan

Day	Task
Day 1	Retake only questions 1-35. Rewrite every missed normalization, join, NULL, and grouping concept.
Day 2	Retake questions 36-72. Write stored procedures and triggers by hand without looking.
Day 3	Timed run: 72 items in 75 minutes, or 60 minutes for extreme pressure. After scoring, create a one-page error log of your weak topics.